NS Estimator – Oracle AE Web Form Plan

Nov 2, 2026 · @Lirik Pre-Sales

Summary

We propose a secure web form where NetSuite (Oracle) AEs submit a customer requirement and get back a reviewed ROM estimate by email as a PDF, in about 1 business day. It reuses the NS Estimator that already runs on AWS. The build takes about 3 weeks and adds about $10–15 a month to running costs.

- **For Oracle AEs:** no Slack account and no login to set up. They fill in one form, confirm their email, and get a PDF they can forward to the customer.
- **For Lirik:** every estimate is still checked and approved by a Lirik presales reviewer in the Slack channel used today. Our rates, hours and pricing logic never leave Lirik.
- **Recommendation:** pilot with 2–5 named NetSuite AEs in the US and Canada, then widen access if the volume and feedback justify it.

Decision requested: approve the 3-week build and the pilot AE list (see Decisions needed at the end).

[How it works for an Oracle AE: end-to-end flow and emails]

What Oracle sees and what stays internal

Oracle gets the same customer-ready content our own AEs see in Slack today; everything that shows how the price is built stays with Lirik.

Item	Oracle AE	Lirik reviewer
ROM price range and currency	Yes	Yes
Duration, phases, payment milestones	Yes	Yes
Scope as understood, what's included	Yes	Yes
Assumptions, exclusions, open questions	Yes	Yes
Exact price, blended rate, hourly rates	No	Yes
Hours, line items, the estimator workbook	No	Yes

Reasons for review, reviewer notes, error details	No	Yes

An automated test produces a sample PDF before each release and fails if a rate, hour count or line item appears in it.

Reliability and security

No request is lost: the form keeps accepting requests and holds them in a queue even when the estimator server is down, and every failure alerts the Lirik team.

Risk	How it is handled
Estimator server down or restarting	Requests wait in the queue for up to 14 days; an alarm fires if any waits over 30 minutes
AI service error or timeout	3 automatic retries, then the AE gets a polite "we're reviewing it manually" email and Lirik is alerted in Slack
Request that keeps failing	Set aside after 3 attempts for manual handling, with an alarm
Same request submitted twice	Detected by submission ID and processed once
Email bounces	Bounce reported back and posted in the review thread
Spam or misuse	Web firewall, CAPTCHA, email confirmation, approved-AE list, limit of 10 requests per AE per day
Misleading text in a requirement	Every answer the AI gives must quote the requirement, and a person approves every estimate
Reviewer slow to respond	Reminder in the review channel after 4 business hours
Server data loss	Daily disk snapshots (needs one AWS permission still to be granted)

Data protection

- All data stays in AWS US East and is encrypted in transit and at rest.
- The server accepts no inbound connections; it only reaches out to fetch queued requests.
- API keys move into AWS Parameter Store and are rotated as part of the build.
- Retention: estimates 12 months, attachments 90 days, unconfirmed submissions 24 hours.
- The form shows a privacy notice and asks the AE to confirm no personal data is included.

Technical design

The new work is only the front door (web form) and the way results go out (email with a PDF). Reading the requirement, pricing the workbook, the review card and the reviewer buttons are unchanged.

Component	AWS service	Purpose
Form page and submit API	API Gateway + 1 Lambda	Serverless, stays up when the EC2 instance is down
Abuse protection	AWS WAF (rate limits, CAPTCHA), API throttling	The form is public
Attachments	Private, encrypted S3 bucket, 90-day expiry	Kept for the reviewer only
Queue between form and bot	SQS + dead-letter queue	Nothing lost during outages
Outgoing email	Amazon SES from estimates@lirik.io (SPF, DKIM, DMARC)	Reliable delivery to Oracle inboxes
PDF	LibreOffice (already in the container)	No new dependency
Monitoring	CloudWatch alarms to the existing SNS topic	Queue age, dead-letter queue, bounces, Lambda errors

Code changes

- New `app/intake.py`: reads the SQS queue and creates requests with source `web` and requester `email:<address>`, then runs the existing pipeline.
- `app/slack_app.py`: delivery, return and failure messages go by email when the request came from the form; failure text for external users is generic.
- `app/slack_ui.py`: review card shows "Web form: name@oracle.com"; the Return dialog warns that the note goes to an external Oracle AE.
- New `app/pdf.py`: builds the PDF from the same data as the requester view.
- New Lambda (about 150 lines): form page, validation, attachment upload, email confirmation.
- Tests: PDF leak test, end-to-end queue test, duplicate-submit test.

Needed from the AWS admin

- Permission to create SQS, SES, Lambda, API Gateway, WAF and one S3 bucket.
- SES domain verification for lirik.io (DNS records) and SES production access (about 1 day).
- Instance role additions: `sqs:ReceiveMessage`, `sqs:DeleteMessage`, `ses:SendRawEmail`, `s3:GetObject` on the attachments bucket, `ssm:GetParameter`.
- DLM or AWS Backup permission for daily snapshots.

Assumptions

The design is sized for a pilot; if any of these change, the plan and cost change with it.

1. **Volume:** 2–5 AEs and under 50 requests a month during the pilot.
2. **Identity:** AEs prove who they are by confirming an email sent to an approved @oracle.com address. No Oracle single sign-on.
3. **Review:** a Lirik reviewer approves every estimate; nothing is sent automatically.
4. **Turnaround:** 1 business day in US hours, with a named backup reviewer.
5. **Coverage:** US and Canada customers, USD or CAD, English only. Other countries are marked as excluded localization.
6. **Attachments:** passed to the reviewer only in the pilot; the AI does not read them yet.
7. **Email delivery:** Oracle's mail system accepts PDF attachments from lirik.io. To be tested with one AE before launch.
8. **Data terms:** an NDA or data-handling agreement with Oracle covers the customer information AEs submit.

9. **Availability:** one server plus the queue is enough; at worst a day of server data could be lost, never a submitted request.
10. **Review tool:** reviewers keep working in Slack; no separate admin website is built.

[Timeline and cost]

[Decisions needed]